Klausur WS 09/10

1 Umrechnung von Zahlensystemen

1.1 Geben Sie die Zahl 0x1f (hexadezimal) in Dezimaldarstellung an.

- Jede Stelle der Zahl hat den Wert der entsprechenden Potenz von 16, die rechte Ziffer entspricht $16^0=1$, die zweite von rechts $16^1=16$ usw. Nimm jede Ziffer bzw. ihren Zahlenwert (A=10, B=11, …) mal mit der entsprechenden Potenz und summiere.
- $1*16^1+15*16^0=1*16+15*1=16+15=31$

1.2 Wie viele Stellen benötigt man zur dezimalen Darstellung der größten Zahl, die 16-stellig noch hexadezimal dargestellt werden kann? (Der Zahlenraum ist auf die positiven Ganzzahlen beschränkt) → Herleitung!

- $\lceil n_{10} \rceil = n_{16} * \frac{\ln 16}{\ln 10} = 16 * \frac{\ln 16}{\ln 10} = 19{,}27 \rightarrow n_{10} = 20$

2 Single Source Shortest Path Problem

2.1 Nennen Sie eine alltegsbezogene SSP-Aufgebenstellung

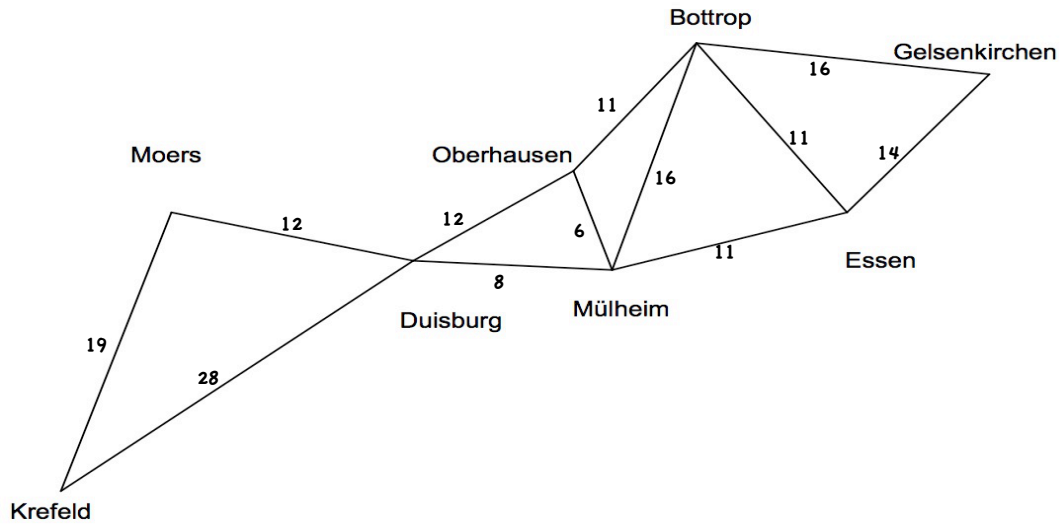
Z.B.: Fahrplan für Züge

2.2 Nennen Sie einen berühmten Informatiker mit Geburtsland, der in Verbindung mit einem Algorithmus zu Lösung des SSP steht.

- Edsger Dijkstra
  - Rotterdem, Holland
  - 1930 - 2002

2.3 Übertragen Sie die Werte der Entfernungstabelle in den Graphen

	Dui	Moe	Kre	Ob	Mü	Bo	Es	Ge
Dui		12	28	12	8			
Moe			19					
Kre								
Ob					6	11		
Mü						16	11	
Bo							11	16
Es								14
Ge								



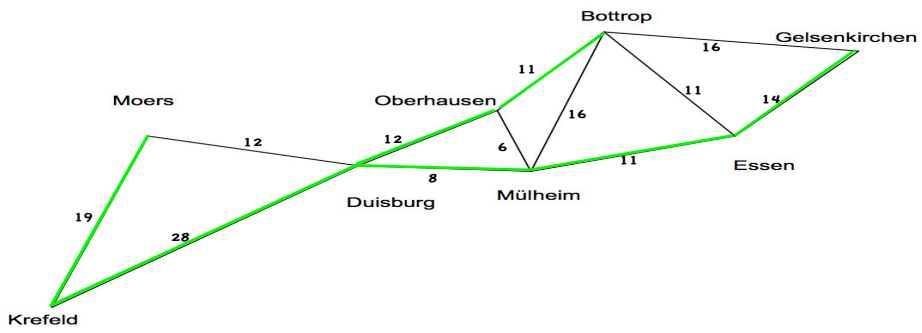
2.4 Lösen Sie das SSP (Startknoten Krefeld) mit Hilfe eines geeigneten Algorithmus aus dem Vorlesungsskript.

Damit die Ausarbeitung nicht zu groß wird, weisen wir den Städten Nummern zu.

1 = Duisburg, 2 = Moers, 3 = Krefeld, 4 = Oberhausen, 5 = Mülheim, 6 = Bottrop,

7 = Essen, 8 = Gelsenkirchen, S = Menge verbundener Knoten,

δ(S) = Menge erreichbarer Knoten mit Entfernung, F = Menge verwendbarer Strecken



S	{3}	Startpunkt
δ(S)	{1 (28), 2 (19)}	Erreichbare Knoten
S	{2, 3}	Wir wählen 2, da kürzeste von 3
F	{(2, 3)}	Stecke kommt dazu
δ(S)	{1 (28)}	Erreichbare Knoten
S	{1, 2, 3}	Wir wählen 1, da kürzeste (einzige)
F	{(1, 3), (2, 3)}	Teilstrecke kommt dazu
δ(S)	{4 (40), 5 (36)}	Neue erreichbare Knoten

S	{1, 2, 3, 5}	Wir wählen 5, da kürzeste von 3
F	{(1, 3), (1, 5), (2, 3)}	Teilstrecke kommt dazu
δ(S)	{4 (40), 6 (52), 7 (47)}	Neue erreichbare Knoten
S	{1, 2, 3, 4, 5}	Wir wählen 4, da kürzeste von 3
F	{(1, 3), (1, 4), (1, 5), (2, 3)}	Teilstrecke kommt dazu
δ(S)	{6 (51), 7 (47), 8 (61)}	Neue kürzeste Entfernung zu 6
S	{1, 2, 3, 4, 5, 7}	Wir wählen 7, da kürzeste von 3
F	{(1, 3), (1, 4), (1, 5), (2, 3), (5, 7)}	Teilstrecke kommt dazu
δ(S)	{6 (51), 8 (61)}	
S	{1, 2, 3, 4, 5, 6, 7}	Wir wählen 6, da kürzeste von 3
F	{(1, 3), (1, 4), (1, 5), (2, 3), (4, 6), (5, 7)}	Teilstrecke kommt dazu
δ(S)	{8 (61)}	
S	{1, 2, 3, 4, 5, 6, 7, 8}	Wir wählen 8, da (letzte) kürzeste von 3
F	{(1, 3), (1, 4), (1, 5), (2, 3), (4, 6), (5, 7), (7, 8)}	Teilstrecke kommt dazu → Fertig!!

2.5 Warum sind in der Entfernungstabelle die dunkelgrauen Felder leer?

Die dunkelgrauen Felder stehen für die Entfernungen von einem Ort zu sich selber, wo logischerweise keine Entfernungen (Werte) enthalten sein können.

2.6 Warum sind in der Entfernungstabelle die hellgrauen Felder leer?

- Es handelt sich um einen ungerichteten Graphen, was zur folge hat, dass es je von einem Ort zu einem Anderen, auch den Rückweg gibt, wenn es denn diesen Weg überhaupt gibt
- Dadurch ist es ausreichend, diese Entfernungen nur ein mal zu beschreiben

2.7 Erklären Sie, wie man anhand der Entfernungstabelle bereits erkennen kann, ob es sich um einen vollvernetzten Graphen handelt!

- Bei einem vollvernetzten Graphen ist jeder Ort mit jedem Anderen verbunden
- Die Tabelle ist entweder ober- und/oder unterhalb der Diagonalen komplett ausgefüllt

2.8 Handelt es sich bei dem Graphen um einen planaren Graphen? Begründen Sie!

Definition 4.11 Lässt sich ein Graph in der Ebene darstellen, ohne dass sich kanten kreuzen, so nenn man ihn planar. Die Menge der durch die Kanten begrenzten Flächenstücke wird mit A bezeichnet.

Satz 4.6 (Eulerformel) Für jeden planaren zusammenhängenden Graphen G = (V, E) gilt

$$|V| - |E| + |A| = 2$$

Nach Satz 4.6:

$|V| - |E| + |A| = 2$

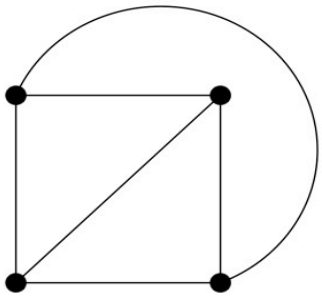
Wobei V die Knoten, E die Kanten und A die Flächenstücken inklusive der kompletten Fläche sind.

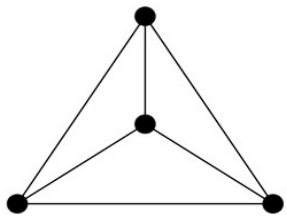
8 – 12 + 6 = 2

→ Vorraussetzung für einen planaren Graphen besteht

→ Folglich handelt es sich im einen planaren Graphen

2.9 Zeichnen Sie einen beliebigen vollvernetzten, planaren Graphen!





3 Asymptotik

3.1 Kreuzen Sie in der folgenden Tabelle alle zutreffenden Felder an, so dass die Landau'schen Symbole eine Verdeutlichung für die Schranke *g*(*n*) zu *f*(*n*) sind. Bespielsweise $f(n)=\omega(g(n))$

$f(n)$	$g(n)$	O	o	Ω	ω	θ
n^2	$\sqrt{n}$			X	X	
$\ln n$	$\log n$	X		X		X
$1/4n^2+3n$	$4n^2+1/3n$	X		X		X
$2n^2+4n$	$n^2-\sin^2 n$	X		X		X
$\log n^2$	$\sin^2 n$	X	X			
$n!$	9^n			X	X	

3.2 Zeigen Sie, dass gilt: $T(n)=\Theta(\ln(5n^2))$

n	10	20	30	40	50
$T(n)$	218 s	281 s	303 s	333 s	340 s
c	35,08	36,97	36,02	37,05	36,04

$$T(n)=c*\ln(5n^2) \quad \rightarrow \quad c=\frac{T(n)}{\ln(5n^2)}$$

Beispiel für $n=50$: $c=\frac{340}{\ln(5*50^2)}=36{,}04$

Bedenkt man allgemeine Abweichungen, liegen die werte doch sehr nah bei einander wodurch man darauf schießen kann, dass die Aussage wahr ist.

4 Automaten / Sprachen / reguläre Ausdrücke

Gegeben sei folgender reguläre Ausdruck der alle Wörter der Sprache S beschreibt: [ab]{1,2} d? A*

4.1 Geben Sie das kürzest mögliche Alphabet ∑ an S an.

$\Sigma = \{a, b, d, A\}$

4.2 Geben Sie alle Wörter der Länge 3 an.

aad, aaA, abd, abA, bad, baA, bbd, bbA, adA, bdA, aAA, bAA

4.3 Ist das Wort „dba“ Bestandteil von ∑*? Begründen Sie!

Ja, da alle benötigten Zeichen im Grundalphabet ∑ vorkommen.

4.4 Gehört das Wort „bdaaaaaaa“ zu S?

Nein, da das kleine a vor dem d kommen muss und höchstens 2 mal (wenn kein b vorhanden wäre) vorkommen darf.

4.5 Konstruieren Sie einen Automaten, der alle Wörter von S erkennt. Beschreiben Sie ihn formal als 5-Tupel. Wählen Sie hierfür eine im Skript verwendete Darstellungsart der Zustandsübergänge, die Ihnen beliebt.

$M = (\{S_1, S_2, Z, T\}, \{a, b, d, A\}, \delta, S, \{S_2, Z, T\})$

	S_1	S_2	Z	T
a	S_2, Z, T	Z, T		
b	S_2, Z, T	Z, T		
d			T	
A				T

5 Sortieralgorithmus: Heap-Sort

5.1 Build-Heap beginnt an der Stelle $i = \lfloor \frac{length\,h(A)}{2} \rfloor$.

Warum nicht an der Stelle i = 1?

Weil sicher gestellt werden muss, dass es darunter nur fertige Heaps gibt und das ist ganz oben eben nicht sichergestellt (sonst wäre der Build-Heap ja nicht nötig).

5.2 Obwohl nach dem Aufruf von Build-Heap bereits sichergestellt ist, dass von Ebene zu Ebene die Elemente absteigend sortiert sind (danach ist ein Nachfolger nie größer als sein Vorgänger), benötigt Heap-Sort anschließend doch etliche Schritte mehr, um die Zahlen komplett zu sortieren. Begründen Sie!

Heapsort muss zum einen die unsortierten Elemente pro Ebene noch sortieren und zum Anderen müssen die Elemente der Wertigkeit nach noch umgedreht werden, da in der obersten Ebene das kleinste Element sein soll und in der Untersten das Größte, nach dem Buils-Heap ist es jedoch umgekehrt.

6 Die folgende Tabelle zeigt die Verteilung einer Hashtabelle mit 5 Feldern:

Feldnummer	Anzahl an Einträgen
1	15
2	5
3	7
4	8
5	4

6.1 Ist bei dieser Tabelle das einfache uniforme Hashing erfüllt? Begründen Sie!

Nein, da beim einfachen uniformen Hashing angenommen wird, dass alle Felder gleich viele Einträge beinhaltet.

6.2 Geben Sie die Auslastung der Tabelle an.

- 39 / 5 → α = 7,8

6.3 Handelt es sich hierbei um offene Adressierung? Begründen Sie!

Nein, da bei einer offenen Adressierung die Auslastung α nicht größer als 1 sein kann. Außerdem dürfte dann nie mehr als ein Element pro Hash-Feld vorkommen.

7 Der Algorithmus „DrehenUndWenden(...)“ nutzt Devide-And-Conquer um eine große Datenmenge abzuarbeiten. Er greift auf die Funktionen „EinfachesDrehen(...)“ und „EinfachesWenden(...)“ zurück.

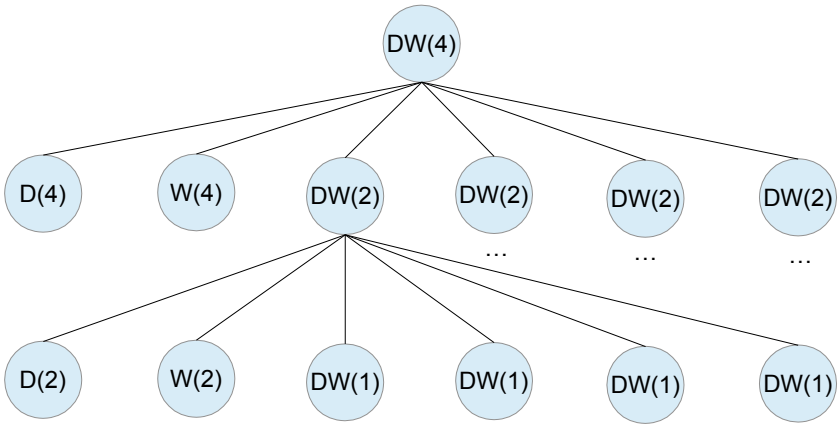
Der Algorithmus hat folgenden Ablauf:

```
DrehenUndWenden (i) {
   if (i > 1) {
      EinfachesDrehen (i);
      EinfachesWenden (i);

      DrehenUndWenden (i/2);
      DrehenUndWenden (i/2);
      DrehenUndWenden (i/2);
      DrehenUndWenden (i/2);
   }
}
```

7.1 Markieren Sie alle rekursiven Aufrufe im obigen Quelltext.

```
DrehenUndWenden (i) {
   if (i > 1) {
      EinfachesDrehen (i);
      EinfachesWenden (i);

      DrehenUndWenden (i/2);
      DrehenUndWenden (i/2);
      DrehenUndWenden (i/2);
      DrehenUndWenden (i/2);
   }
}
```

7.2 Zeichnen Sie den Rekursionsbaum für DrehenUndWenden(4). Kürzen Sie hierbei die Funktionsnamen „EinfachesDrehen“ mit „D“, „EinfachesWenden“ mit „W“ und „DrehenUndWenden“ mit „DW“ ab.



7.3 Komläxitätsberechnung eines Algorithmus

Folgende Rechenzeiten sind bestimmt worden:

$$T_{EinfachesDrehen}(n) = \Theta(n^2)$$
$$T_{EinfachesWenden}(n) = \Theta(n^2)$$

Nutzen Sie die Kenntnis, dass der Algorithmus rekursiv ist (Mastertheorem!) zur Berechnung von $T_{DrehenUndWenden}(n)$!

$$T(n) = 4\mathrm{T}(\frac{n}{2}) + \Theta(n^2)$$
$$a = 4, \quad b = 2, \quad f(n) = \Theta(n^2)$$
$$\Theta(n^{\log_2 4}) = \Theta(n^2) \quad = \quad f(n) = \Theta(n^2)$$
Mastertheorem 2. Fall: $T(n) = \Theta(n^{\log_2 4} \log n) = \Theta(n^2 \log n)$